

云计算答辩

 答辩学生：王公泽  指导老师：葛强

15组

选题的背景意义

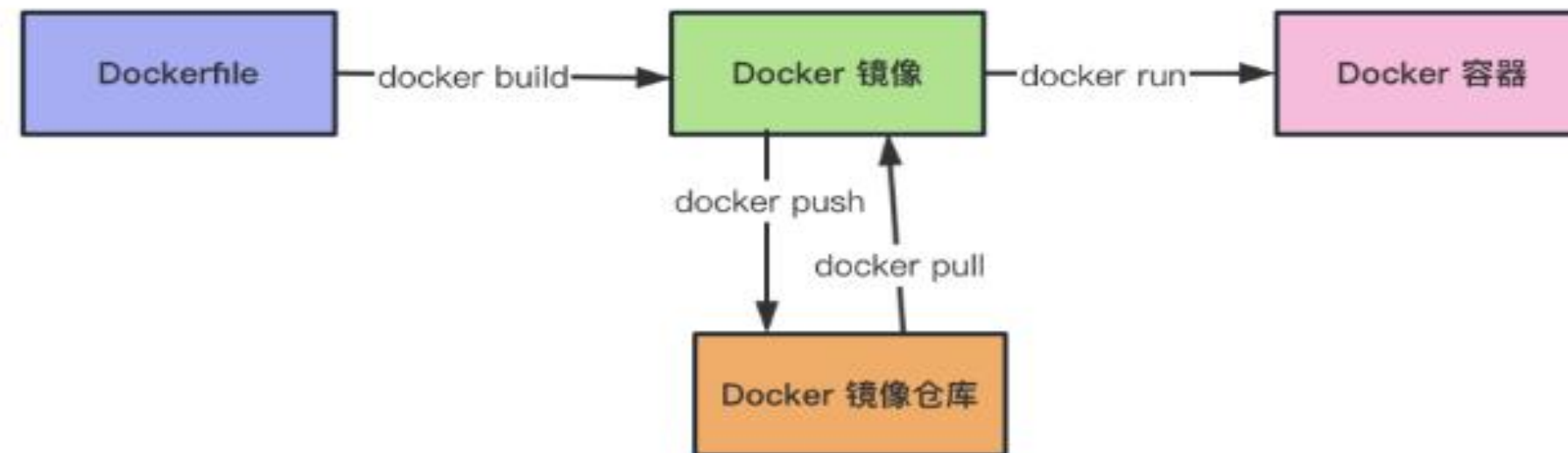


选题背景

在云计算越来越普及的当下，无论是初创公司的轻量业务，还是大型企业的复杂系统，都在依赖云端的弹性算力、分布式存储快速迭代——无需自建机房，只需通过网络就能调用全球范围内的IT资源，让业务落地效率翻倍。但随着应用架构日趋复杂，跨环境部署的“痛点”也逐渐凸显：开发环境、测试环境、云端生产环境的配置差异，常常导致“本地能跑，云端报错”的尴尬；传统虚拟机部署占用资源多、启动慢，也让云端资源的利用率难以最大化。此时，Docker作为容器化技术的核心工具，恰好成为云计算的“最佳拍档”，完美补齐了这些短板。

题目 6（Docker）镜像构建

Dockerfile 是用于构建 Docker 镜像的纯文本配置文件，本质是镜像构建的“自动化说明书”——它通过一系列标准化指令（如 FROM 指定基础环境、RUN 安装依赖、COPY 复制代码、CMD 定义启动命令等），按顺序定义镜像的构建步骤，能让 Docker 自动、标准化地搭建应用运行环境并打包应用，实现“一次编写，处处构建”，彻底解决跨环境部署的一致性问题。



项目结构

Dockerfile

构建原始镜像的“蓝图”，定义了从基础镜像、安装依赖到启动应用的完整流程

Dockerfile.slim

瘦身版镜像的构建文件，通过多阶段构建仅保留运行必需内容，大幅减小镜像体积

requirements.txt

Python 依赖清单，列出 Flask 等应用运行所需的包，供 Dockerfile 安装使用

app.py

Flask 应用的源代码，是实际提供服务的业务程序

manage-flask.sh

一键管理脚本，集成了构建镜像、启动 / 停止容器、查看日志等常用操作

requirements.txt

```
1 flask==2.3.3
```

requirements.txt是Python 项目的依赖配置文件，明确指定当前项目需要安装的 Python 包及精确版本。

app.py

```
1  from flask import Flask
2  import os
3  import logging
4  from datetime import datetime
5
6  app = Flask(__name__)
7
8  APP_NAME = os.getenv("APP_NAME", "DefaultApp")
9  PORT = int(os.getenv("PORT", 5000))
10 LOG_LEVEL = os.getenv("LOG_LEVEL", "INFO")
11 LOG_DIR = "/app/logs"
12
13 logging.basicConfig(
14     level=LOG_LEVEL,
15     format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
16     handlers=[
17         logging.FileHandler(f"{LOG_DIR}/app_{datetime.now().strftime('%Y%m%d')}.log"),
18         logging.StreamHandler()
19     ]
20 )
21 logger = logging.getLogger(APP_NAME)
22
23 @app.route("/")
24 def index():
25     msg = f"Hello from {APP_NAME}! Running on port {PORT}"
26     logger.info(msg)
27     return msg
28
29 if __name__ == "__main__":
30     app.run(host="0.0.0.0", port=PORT)
```

这段代码的运行过程可简化为以下步骤：

容器启动触发执行：

基于包含该代码的 Docker 镜像启动容器后，容器内执行python app.py，代码开始运行；

初始化与配置：

先创建 Flask 应用实例，读取环境变量（应用名、端口等，无则用默认值），再配置日志系统（按日期生成日志文件，日志同时输出到文件和容器终端）；

启动 Web 服务：

绑定0.0.0.0（允许容器外访问）和指定端口，启动 Flask 服务，持续监听端口等待请求；

处理请求：

当外部访问容器映射的端口 / 根路径（/）时，执行路由函数，生成并返回包含应用名、端口的提示信息，同时记录 INFO 级日志；

持续运行：

服务保持运行状态，持续监听并处理后续请求，日志实时写入文件 + 输出到终端（可通过docker logs查看）。

Dockerfile

这个 Dockerfile 的作用是构建一个轻量的 Python Flask 应用 Docker 镜像，核心流程是标准化配置应用的运行环境并打包应用，具体功能可概括为：

基础环境搭建：

基于轻量的python:3.9-alpine镜像（节省体积），设置工作目录为/app，并配置维护者元信息；

环境与依赖配置：

定义应用所需的环境变量（应用名、端口、日志级别），复制依赖文件requirements.txt并安装Python 依赖；

应用部署：

复制 Flask 应用代码app.py到镜像中，创建日志目录并赋予写入权限；

容器配置：

暴露应用端口（5000），定义启动命令（通过sh -c执行python app.py），确保容器启动时能直接运行该 Flask 应用。

```
1 FROM python:3.9-alpine
2
3 LABEL maintainer="xjn@xxt.com"
4
5 WORKDIR /app
6
7 ENV APP_NAME="MyFlaskApp" \
8     PORT=5000 \
9     LOG_LEVEL="INFO"
10
11 COPY requirements.txt .
12 RUN pip install --no-cache-dir -r requirements.txt
13
14 COPY app.py .
15
16 RUN mkdir -p /app/logs && chmod 777 /app/logs
17
18 EXPOSE $PORT
19
20 CMD ["sh", "-c", "python app.py"]
```

Dockerfile.slim

这个 Dockerfile 采用多阶段构建方式，核心作用是构建体积更轻量的 Python Flask 应用镜像，既保证依赖安装完整，又能大幅减小最终镜像体积，具体功能拆解：

阶段 1：构建依赖（builder 阶段）

以相对完整的python:3.9-slim为基础镜像，设置工作目录/app；复制requirements.txt，并将 Python 依赖（如 Flask）安装到指定目录/app/deps（--no-cache-dir避免缓存，进一步减小体积）。

阶段 2：构建最终镜像（轻量阶段）

切换到更轻量的python:3.9-alpine（Alpine 是极小的 Linux 发行版，能大幅压缩镜像体积）；

从 builder 阶段复制已安装的依赖目录/app/deps到当前镜像，避免在 Alpine 中重复安装依赖（减少构建时间 + 镜像体积）；复制应用代码app.py，配置PYTHONPATH让 Python 能识别依赖；定义应用环境变量、创建日志目录并授权、暴露端口，最后指定启动命令启动 Flask 服务。

```
1 FROM python:3.9-slim AS builder
2 WORKDIR /app
3
4 COPY requirements.txt .
5 RUN pip install --no-cache-dir -r requirements.txt -t /app/deps
6
7 FROM python:3.9-alpine
8 WORKDIR /app
9
10 COPY --from=builder /app/deps /app/deps
11 COPY app.py .
12
13 ENV PYTHONPATH=/app/deps
14 ENV APP_NAME="SlimFlaskApp" PORT=5000
15
16 RUN mkdir -p /app/logs && chmod 777 /app/logs
17 EXPOSE $PORT
18
19 CMD ["python", "app.py"]
```


manage-flask.sh

```
1  #!/bin/bash
2
3
4  # --- 配置区 ---
5  # 镜像和容器名称
6  IMAGE_NAME="flask-demo"
7  TAG_ORIG="v1.0"
8  TAG_SLIM="slim"
9  CONTAINER_NAME="flask-app"
10
11 # 端口映射 (主机端口:容器端口)
12 PORT_MAP="8080:5000"
13
14 # 卷挂载 (主机目录:容器目录)
15 # 确保主机目录存在
16 HOST_LOG_DIR="$HOME/docker-demo/host-logs"
17 CONTAINER_LOG_DIR="/app/logs"
18 VOLUME_MAP="${HOST_LOG_DIR}:${CONTAINER_LOG_DIR}"
19
20 # 环境变量
21 ENV_VARS="-e APP_NAME='MyAwesomeApp' -e PORT=5000"
```

简单定义了一下各种变量，
主要包括镜像和容器名称，端口映射，
目录环境变量

manage-flask.sh

```
23 # --- 脚本主逻辑 ---
24 case "$1" in
25     build)
26         echo "===== 开始构建原始镜像: ${IMAGE_NAME}:${TAG_ORIG} ====="
27         docker build -t "${IMAGE_NAME}:${TAG_ORIG}" .
28
29         echo -e "\n===== 开始构建瘦身镜像: ${IMAGE_NAME}:${TAG_SLIM} ====="
30         if [ -f "Dockerfile.slim" ]; then
31             docker build -f "Dockerfile.slim" -t "${IMAGE_NAME}:${TAG_SLIM}" .
32
33             echo -e "\n===== 镜像体积对比 ====="
34             ORIG_SIZE=$(docker inspect -f '{{.Size}}' "${IMAGE_NAME}:${TAG_ORIG}" 2>/dev/null || echo 0)
35             SLIM_SIZE=$(docker inspect -f '{{.Size}}' "${IMAGE_NAME}:${TAG_SLIM}" 2>/dev/null || echo 0)
36
37             if [ "$ORIG_SIZE" != "0" ] && [ "$SLIM_SIZE" != "0" ]; then
38                 ORIG_MB=$(echo "scale=2; $ORIG_SIZE/1024/1024" | bc)
39                 SLIM_MB=$(echo "scale=2; $SLIM_SIZE/1024/1024" | bc)
40                 REDUCE_MB=$(echo "scale=2; $ORIG_MB - $SLIM_MB" | bc)
41                 REDUCE_PERCENT=$(echo "scale=2; ($REDUCE_MB/$ORIG_MB)*100" | bc)
42
43                 echo "原始镜像体积: ${ORIG_MB} MB"
44                 echo "瘦身镜像体积: ${SLIM_MB} MB"
45                 echo "体积减少:    ${REDUCE_MB} MB (${REDUCE_PERCENT}%)"
46             else
47                 echo "无法计算体积, 可能有镜像未成功构建。"
48             fi
49         else
50             echo "错误: 未找到 'Dockerfile.slim' 文件, 跳过瘦身镜像构建。"
51         fi
52     ;;
```

如果后续输入有bulid，则会构建两个镜像，且对比他们瘦身前和瘦身后的的体积

```
===== 镜像体积对比 =====
原始镜像体积: 58.94 MB
瘦身镜像体积: 52.72 MB
体积减少:    6.22 MB (10.00%)
```

manage-flask.sh

```
54 start)
55 # 检查容器是否已存在，存在则先删除
56 if [ "$(docker ps -aq -f name=${CONTAINER_NAME})" ]; then
57     echo "容器 ${CONTAINER_NAME} 已存在，正在删除..."
58     docker rm -f "${CONTAINER_NAME}"
59 fi
60
61 # 确保主机日志目录存在
62 mkdir -p "${HOST_LOG_DIR}"
63
64 echo "==== 启动容器: ${CONTAINER_NAME} ====="
65 echo " - 镜像: ${IMAGE_NAME}:${TAG_SLIM}"
66 echo " - 端口: ${PORT_MAP}"
67 echo " - 卷挂载: ${VOLUME_MAP}"
68 echo " - 环境变量: ${ENV_VARS}"
69
70 # 使用 eval 来正确处理包含空格的环境变量
71 eval "docker run -d \
72     --name ${CONTAINER_NAME} \
73     -p ${PORT_MAP} \
74     -v ${VOLUME_MAP} \
75     ${ENV_VARS} \
76     ${IMAGE_NAME}:${TAG_SLIM}"
77
78 echo -e "\n容器已启动!"
79 echo "访问地址: http://localhost:${PORT_MAP%:*}"
80 echo "查看日志: ./manage-flask.sh logs"
81 ;;
```

如果后续输入有start，则会先检测容器是否存在，存在就先删除，如果不存在才会挂载并启动，就可以访问页面

```
(base) hadoop@master:~/docker-demo$ ./manage-flask.sh start
==== 启动容器: flask-app ====
- 镜像: flask-demo:slim
- 端口: 8080:5000
- 卷挂载: /home/hadoop/docker-demo/host-logs:/app/logs
- 环境变量: -e APP_NAME='MyAwesomeApp' -e PORT=5000
9b52e16d952824a9e1eef Fad2303775230e2156f3769150d32a466ad6890ad87

容器已启动!
访问地址: http://localhost:8080
查看日志: ./manage-flask.sh logs
```


manage-flask.sh

```
stop)
    echo "==== 停止并删除容器: ${CONTAINER_NAME} ====="
    docker stop "${CONTAINER_NAME}"
    docker rm "${CONTAINER_NAME}"
    echo "操作完成。"
    ;;

logs)
    echo "==== 实时查看容器日志 (按 Ctrl+C 退出) ====="
    docker logs -f "${CONTAINER_NAME}"
    ;;

status)
    echo "==== 容器 ${CONTAINER_NAME} 状态 ====="
    docker ps -a --filter "name=${CONTAINER_NAME}" --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
    ;;

*)
    echo "使用方法: $0 {build|start|stop|logs|status}"
    echo "  build   - 构建原始和瘦身镜像, 并对比体积"
    echo "  start   - 启动容器 (使用瘦身镜像)"
    echo "  stop    - 停止并删除容器"
    echo "  logs    - 实时查看容器日志"
    echo "  status  - 查看容器状态"
    exit 1
    ;;
esac

exit 0
```

如果后续输入有stop, 则会停止并删除容器
如果后续输入有logs, 则会查看日志
如果后续不符合任何上述输入, 则弹出指令帮助

最终结果

